Customer Voice Intelligence Agent

Set A - AI Catalyst Program | Havells India Ltd.

An Agentic AI System for Automated Review Analysis,
Theme Discovery, and Grounded Product Intelligence

Built with: Python | Scikit-learn | Multi-Agent Architecture | RAG

GitHub Repository: https://github.com/Spiderboyis/Havelles

1. Executive Summary

This document presents a production-grade agentic AI system that transforms raw, messy customer reviews of Havells consumer appliances into actionable product intelligence. The system automatically identifies recurring complaint themes, tracks sentiment shifts over time, and answers product manager queries in plain English - with every answer grounded in actual review data.

Key capabilities: (1) Aspect-Based Sentiment Analysis across 10 product dimensions, (2) Unsupervised theme discovery via TF-IDF + KMeans clustering, (3) Temporal trend detection with seasonal pattern awareness, (4) Grounded Q&A with source citations, (5) Multi-agent orchestration with intent-based routing.

Problem Addressed

Havells produces fans, water heaters, air purifiers, mixer grinders, and lighting products. Customer reviews pile up faster than teams can read. Product managers need to know: What are people unhappy about? On which product? Is it getting better or worse? This system answers these questions automatically, honestly, and with evidence.

2. System Architecture

The system follows the Orchestrator-Worker multi-agent pattern. A central Orchestrator agent receives the manager's question, classifies intent, and routes to specialist agents. Results are aggregated by a Synthesizer agent that ensures grounding.

Architecture Diagram (Text)

```
  Manager Query
       |
  [Orchestrator Agent] -- classifies intent, extracts filters
       |
  +----+-----+----------+-----------+
  |          |          |           |
  v          v          v           v
[Retrieval] [Sentiment] [Theme]  [Trend]
  Agent      Agent      Agent     Agent
  |          |          |           |
  +----+-----+----------+-----------+
       |
  [Synthesizer Agent] -- grounds & cites
       |
  Grounded Response + Citations
```

Agent Descriptions

Agent	Role
Orchestrator	Intent classification via regex patterns. Routes to specialists. No LLM needed.
Retrieval Agent	TF-IDF vector search over indexed reviews. Supports filtering.
Sentiment Agent	Lexicon-based ABSA with negation handling, intensifiers, emotion detection.
Theme Agent	TF-IDF + KMeans clustering for unsupervised theme discovery.
Trend Agent	Temporal aggregation computing monthly sentiment trends.
Synthesizer	Aggregates outputs, validates grounding, generates cited response.

3. Data Pipeline

3.1 Data Generation

We built a synthetic review generator producing 2,500 realistic reviews across 5 product categories (fans 30%, water heaters 22%, mixer grinders 20%, lighting 16%, air purifiers 12%). Reviews include temporal patterns (seasonal complaint spikes), product-specific issues, varied writing styles, and Indian English patterns.

3.2 Ingestion Pipeline

- Text cleaning: URL/email removal, punctuation normalization, whitespace cleanup
- Feature extraction: Aspect keyword detection across 10 product dimensions
- Signal detection: Complaint/praise keyword matching for pre-filtering
- Deduplication: MD5 hash-based duplicate removal
- Validation: Minimum word count filtering, encoding normalization

3.3 Data Flow

```
Raw Reviews (JSON/CSV)
  -> DataIngestionPipeline.process_batch()
     -> clean_text() -> extract_aspects() -> detect_signals()
  -> ProcessedReview objects
  -> VectorStoreManager.index_reviews()
  -> TF-IDF Matrix (ready for retrieval)
```

4. Core Components Deep-Dive

4.1 Sentiment Analysis Engine

Hybrid approach combining domain-specific lexicon (80+ positive/negative terms with scores from -1 to +1) with context-aware features:

- Negation handling: 'not good' correctly flips to negative (partial inversion at 0.75x)
- Intensifiers/diminishers: 'very good' = 1.3x, 'slightly bad' = 0.7x
- Aspect-Based SA: 10 aspects (performance, noise, service, safety, etc.) with aspect-specific context words
- Emotion detection: Frustration, satisfaction, anger, trust, disappointment

Design Decision: We use lexicon-based rather than LLM-based sentiment because it provides deterministic, reproducible, auditable results at batch speed. LLM is reserved for the synthesis layer where natural language generation quality matters.

4.2 Theme Discovery Engine

Unsupervised theme extraction using TF-IDF vectorization + KMeans clustering:

- TF-IDF with 3000 features, (1,2)-gram range, sublinear TF weighting
- KMeans with 10-12 clusters, 10 random initializations
- Auto-labeling: Maps top cluster keywords to 16 predefined domain theme labels
- Per-theme metrics: Sentiment distribution, category breakdown, temporal trend
- Representative review selection via cosine similarity to cluster centroid

4.3 Vector Store & Retrieval

Lightweight TF-IDF vector store for grounded retrieval. Reviews are enriched with metadata (product name, category, rating-based sentiment words) before indexing. Retrieval supports category, rating range, and date filters. For production at scale, this swaps to ChromaDB or FAISS with sentence-transformer embeddings.

5. Grounding & Faithfulness

The non-negotiable requirement: every answer must be grounded in actual review data. Our grounding strategy operates at three levels:

Level 1: Retrieval Grounding

All analysis operates ONLY on retrieved reviews. The system never generates insights from parametric knowledge. If fewer than 5 reviews are retrieved, a data limitation warning is appended to the response.

Level 2: Evidence Citations

Every response includes SOURCE CITATIONS with review ID, product name, rating, date, and a verbatim snippet. Users can trace any claim back to specific reviews.

Level 3: Honest Uncertainty

The system explicitly states data coverage (e.g., 'Based on 20 relevant reviews'). When data is insufficient for a conclusion, it says so rather than inventing one. A confidence score (0-1) is computed based on retrieval coverage.

Faithfulness Evaluation

The EvaluationFramework checks: (1) Citation presence, (2) Data coverage mention, (3) Limitation acknowledgment, (4) Quote grounding rate - verifying quoted text against source reviews. These produce an overall faithfulness score.

6. Query Processing & Intent Classification

The Orchestrator classifies queries into 7 intent categories using regex pattern matching (no LLM needed for routing, keeping it fast and deterministic):

Intent	Example	Agent Pipeline
Sentiment Overview	'How do customers feel about fans?	Retrieval -> Sentiment -> Synthesizer
Theme Discovery	'What are main complaints?'	Retrieval -> Theme -> Synthesizer
Trend Analysis	'Is satisfaction improving?'	Retrieval -> Sentiment -> Trend -> Synthesizer
Product Comparison	'Compare fans vs heaters'	Retrieval -> Sentiment -> Theme -> Synthesizer
Specific Issue	'Tell me about noise issues'	Retrieval -> Sentiment -> Theme -> Synthesizer
Aspect Deep-Dive	'How is build quality?'	Retrieval -> Sentiment -> Synthesizer
General Summary	'Give me an overview'	All agents

Filter Extraction

The Orchestrator also extracts contextual filters from the query: product category (fans, water_heaters, etc.), aspect (noise, service, quality), and date ranges. These filters are passed to the Retrieval Agent for focused search.

7. Evaluation Framework

The system is evaluated across four dimensions:

7.1 Retrieval Quality

Category-level precision: For category-specific queries, what percentage of retrieved reviews belong to the correct category. Target: >80%.

7.2 Sentiment Accuracy

Ternary classification accuracy (positive/neutral/negative) measured against synthetic data ground truth. Per-class precision, recall, and F1 scores are computed.

7.3 Theme Coherence

Theme distinctness (1 - avg Jaccard overlap between keyword sets) and size uniformity (entropy-based measure of cluster balance). High distinctness = non-overlapping themes.

7.4 Grounding Faithfulness

Citation presence, data coverage mention, limitation acknowledgment, and quote grounding rate. Combined into an overall faithfulness score (0-1).

8. Scaling to Full Catalogue

The problem statement asks: what would change if watching the whole catalogue? Here is our scaling roadmap:

Data Layer

- Replace TF-IDF vectors with sentence-transformer dense embeddings (384-dim)
- Swap in-memory store for ChromaDB/Pinecone with metadata filtering
- Add Apache Kafka / Redis Streams for real-time review ingestion
- Implement incremental indexing (add new reviews without full re-index)

Compute Layer

- Parallelize agent execution (sentiment + theme can run concurrently)
- Cache frequent queries and pre-compute category-level summaries daily
- Use async processing for LLM calls when synthesis uses GPT-4

Agent Layer

- Add a Scraping Agent for automated review collection from Amazon/Flipkart
- Add an Alerting Agent for detecting sudden sentiment drops (anomaly detection)
- Add a Comparison Agent for competitive analysis (Crompton, Bajaj, Orient)
- Implement LangGraph for stateful multi-turn conversations with checkpointing

Quality Layer

- A/B testing framework for comparing analysis approaches
- Human-in-the-loop feedback for theme label validation
- Continuous evaluation dashboard tracking all 4 quality dimensions

9. Technology Stack

Component	Technology
Language	Python 3.10+
NLP/ML	Scikit-learn (TF-IDF, KMeans), NumPy
Sentiment	Custom lexicon engine with ABSA
Topic Modeling	TF-IDF + KMeans (BERTopic-ready)
Vector Search	TF-IDF cosine similarity (ChromaDB-ready)
Agent Framework	Custom orchestrator (LangGraph-ready)
LLM (optional)	OpenAI GPT-4o-mini for synthesis
PDF Generation	fpdf2
Data Format	JSON, CSV

10. Project Structure

```
havells/
  main.py                    # Entry point
  requirements.txt           # Dependencies
  config/settings.py         # Centralized config
  src/
    agents/orchestrator.py   # Multi-agent system
    core/
      data_ingestion.py      # Preprocessing pipeline
      sentiment_engine.py    # ABSA engine
      theme_engine.py        # Topic discovery
      vector_store.py        # Retrieval system
      evaluation.py          # Quality metrics
    data/review_generator.py # Synthetic data
    utils/pdf_generator.py   # This report
  data/                      # Generated datasets
  output/                    # Reports & PDFs
```

11. Key Design Decisions

Lexicon over LLM for sentiment
Rationale: Deterministic, fast, auditable. LLM reserved for synthesis only.

TF-IDF over transformers for retrieval
Rationale: Zero GPU dependency. Swappable to dense embeddings for production.

Regex intent classification
Rationale: Fast, transparent routing. No LLM latency for query classification.

Stateless agents with explicit state

Rationale: Each agent is a pure function. No hidden state, easy to test and debug.

Synthetic data with realistic patterns

Rationale: Seasonal trends, Indian English, product-specific issues enable proper evaluation.

Grounding-first architecture

Rationale: Every response cites sources. System admits uncertainty rather than hallucinating.

12. Conclusion

This Customer Voice Intelligence Agent demonstrates a practical, production-oriented approach to automated review analysis. Unlike polished demos that hide complexity, this solution exposes the full design thinking: how data flows through the system, how work is split across specialized agents, how information passes between components, how output honesty is enforced through grounding, how quality is measured through a four-dimensional evaluation framework, and what would need to change at catalogue scale.

The system processes 2,500 reviews across 5 Havells product categories, discovers recurring themes without labeled data, tracks sentiment trends over 18 months, and answers product manager questions with cited evidence. Every claim is traceable to source reviews, and every limitation is acknowledged.

The architecture is designed for incremental enhancement: swap TF-IDF for transformers, add LangGraph for multi-turn conversations, integrate real-time scraping - each change is isolated to a single component while the agent coordination layer remains stable.